

Project 1: Numerical Integrators for a Coupled Oscillator and a Stopping Distance Under Drag

Javad Yousefian

September 28, 2026

Abstract

In this project I wrote numerical integrators for two physics problems and tested them against their exact solutions. The first problem is an ODE, a two-mass three-spring coupled oscillator, where I wrote forward Euler and 4th order Runge-Kutta (RK4) by hand. The second problem is a definite integral, the distance travelled by an object slowing down under linear drag, where I wrote the left Riemann sum, the trapezoidal rule, and Simpson's rule by hand. I also compared my code to the scipy versions. The errors I get go like h^1 for Euler, h^4 for RK4, h^1 for Riemann, h^2 for trapezoid, and h^4 for Simpson, which is what theory says. I also checked two physical properties for each problem. My code is worst for Euler on the oscillator, where the energy grows a lot with time.

1 Introduction

The goal of this project is to build small numerical integrators, test them against problems with known exact answers, and check that the errors follow what the theory predicts. I picked problems that have closed-form solutions on purpose, because that way any difference between my code and the exact answer is a real error I can measure.

Section 2 sets up the two problems and gives the exact solutions. Section 3 describes the five integrators I wrote. Section 4 discusses the truncation errors. Section 5 briefly explains how the code files are organized. Section 6 presents the validation of my code. Section 7 explains where my code is worst.

2 Physics Problems and Exact Solutions

2.1 Two-mass, three-spring coupled oscillator (ODE)

I take two equal masses m that can slide on a track without friction. They are connected to each other and to two walls by three springs, all with the same stiffness k (wall- m_1 - m_2 -wall). I call x_1 and x_2 the displacements of the two masses from their equilibrium positions.

To find the equations of motion I look at which springs are pulling on each mass. The left spring pulls mass 1 back with force $-kx_1$. The middle spring is stretched by $x_2 - x_1$, so it pulls mass 1 with $+k(x_2 - x_1)$ and mass 2 with the opposite force $-k(x_2 - x_1)$. The right spring pulls mass 2

back with $-kx_2$. Using Newton's second law:

$$m\ddot{x}_1 = -kx_1 + k(x_2 - x_1) = -2kx_1 + kx_2, \quad (1)$$

$$m\ddot{x}_2 = -k(x_2 - x_1) - kx_2 = kx_1 - 2kx_2. \quad (2)$$

These are two second-order equations, but the integrators need a first-order system. So I use the state $\mathbf{y} = (x_1, x_2, v_1, v_2)$ and add $\dot{x}_i = v_i$ to the two equations above.

To find the normal modes I try solutions of the form $x_1, x_2 \propto e^{i\omega t}$. This gives two frequencies:

$$\omega_+ = \sqrt{k/m} \quad (\text{in-phase}), \quad \omega_- = \sqrt{3k/m} \quad (\text{out-of-phase}). \quad (3)$$

The in-phase mode is when both masses move together ($x_1 = x_2$), so the middle spring is never stretched and only the two outer springs act. That is why the frequency is just $\sqrt{k/m}$. The out-of-phase mode is when they move against each other ($x_1 = -x_2$), so the middle spring is stretched twice as much and the frequency is bigger.

Any initial condition can be written as a combination of the two normal modes. So I have a closed-form solution for $x_1(t)$ and $x_2(t)$ that I use to check my integrators. In practice I project (x_1, x_2, v_1, v_2) onto the normal-mode coordinates $q_{\pm} = (x_1 \pm x_2)/\sqrt{2}$, evolve each one with cos and sin at frequency $\omega_{\pm}$, and rotate back.

The total mechanical energy is the kinetic energy of the two masses plus the potential energy of the three springs:

$$E = \frac{1}{2}m(v_1^2 + v_2^2) + \frac{1}{2}k[x_1^2 + (x_2 - x_1)^2 + x_2^2]. \quad (4)$$

There is no friction, so E should stay constant in time.

2.2 Stopping distance under linear drag (integral)

Now I take an object of mass m moving in one dimension with initial speed v_0 , with only a linear drag force $F = -bv$. Newton's second law gives $m\dot{v} = -bv$. I can rewrite this as $dv/v = -(b/m)dt$. Integrating both sides and using $v(0) = v_0$ gives

$$v(t) = v_0 e^{-t/\tau}, \quad \tau = m/b, \quad (5)$$

where τ is the time it takes for the speed to drop by a factor of e . The distance travelled between $t = 0$ and $t = T$ is

$$x(T) = \int_0^T v_0 e^{-t/\tau} dt. \quad (6)$$

Doing this integral by hand (the antiderivative is $-v_0\tau e^{-t/\tau}$) gives

$$x(T) = v_0\tau (1 - e^{-T/\tau}). \quad (7)$$

3 Numerical Methods

I wrote each method as a small standalone Python function.

3.1 ODE integrators

For a first-order system $\mathbf{y}' = f(t, \mathbf{y})$ starting from $\mathbf{y}(t_n) = \mathbf{y}_n$, one step of size h is:

$$\text{Euler: } \mathbf{y}_{n+1} = \mathbf{y}_n + h f(t_n, \mathbf{y}_n), \quad (8)$$

$$\begin{aligned} \text{RK4: } \mathbf{k}_1 &= f(t_n, \mathbf{y}_n), \\ \mathbf{k}_2 &= f(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_1/2), \\ \mathbf{k}_3 &= f(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_2/2), \\ \mathbf{k}_4 &= f(t_n + h, \mathbf{y}_n + h\mathbf{k}_3), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \end{aligned} \quad (9)$$

The RK4 weights $(1, 2, 2, 1)/6$ are the classical values used in the standard RK4 method.

For the comparison I also used `scipy.integrate.solve_ivp` with the “DOP853” method at tight tolerances (10^{-10} and 10^{-12}). At those tolerances `scipy` is much more accurate than my code, so I use it as a very accurate numerical reference to check against, together with the exact solution.

3.2 Quadrature rules

For $I = \int_a^b f(x)dx$ on n sub-intervals of width $h = (b - a)/n$ with grid points $x_i = a + ih$, my three rules are:

$$I_{\text{Riemann}} = h \sum_{i=0}^{n-1} f(x_i), \quad (10)$$

$$I_{\text{trap}} = h \left[\frac{1}{2}f(x_0) + \sum_{i=1}^{n-1} f(x_i) + \frac{1}{2}f(x_n) \right], \quad (11)$$

$$I_{\text{Simpson}} = \frac{h}{3} \left[f(x_0) + f(x_n) + 4 \sum_{i \text{ odd}} f(x_i) + 2 \sum_{i \text{ even, interior}} f(x_i) \right]. \quad (12)$$

Simpson’s rule needs n to be even, my code raises an error if it is not. I also compared to `scipy.integrate.trapezoid` and `scipy.integrate.simpson`, which use the same formulas on the same grid.

4 Truncation Error

The truncation error is the error left over when I compare one step of the method to the exact update. In every case below I use Taylor expansion of the exact solution around t_n (or of $f(x)$ around x_i), plug the method’s formula into that, and see which power of h is the first one that does not cancel. The local error is what is left after one step. The global error is what you get after $N = (t_{\text{end}} - t_0)/h$ steps, which usually loses one power of h because N is proportional to $1/h$.

4.1 Euler

Taylor-expand the exact solution around t_n :

$$\mathbf{y}(t_n + h) = \mathbf{y}_n + h\mathbf{y}'_n + \frac{h^2}{2}\mathbf{y}''_n + O(h^3). \quad (13)$$

Euler's update is $\mathbf{y}_{n+1} = \mathbf{y}_n + hf(t_n, \mathbf{y}_n) = \mathbf{y}_n + h\mathbf{y}'_n$, which keeps only the first two terms of the expansion. Subtracting Euler's update from the exact one, the local error (error in one step) is

$$\tau_{\text{loc}}^{\text{Euler}} = \frac{h^2}{2}\mathbf{y}''(t_n) + O(h^3) = O(h^2). \quad (14)$$

For a fixed interval $[t_0, t_{\text{end}}]$ I take $N = (t_{\text{end}} - t_0)/h$ steps, each contributing $O(h^2)$. So the global error is

$$\tau_{\text{glob}}^{\text{Euler}} = N \cdot O(h^2) = \frac{t_{\text{end}} - t_0}{h} \cdot O(h^2) = O(h). \quad (15)$$

4.2 RK4

I do not derive the RK4 weights, but I can show why the error is $O(h^5)$ locally. The idea is that each of the four slope evaluations $\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3, \mathbf{k}_4$ is a Taylor-expanded value of f at a slightly different $(t, \mathbf{y})$. When I plug them into the combination

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

and expand everything in h , the coefficients $(1, 2, 2, 1)/6$ are chosen exactly so that the terms of order h, h^2, h^3, h^4 match the Taylor expansion of the exact solution

$$\mathbf{y}(t_n + h) = \mathbf{y}_n + h\mathbf{y}'_n + \frac{h^2}{2}\mathbf{y}''_n + \frac{h^3}{6}\mathbf{y}'''_n + \frac{h^4}{24}\mathbf{y}^{(4)}_n + O(h^5). \quad (16)$$

So the first term that does not cancel is $O(h^5)$, which gives the local error. The global error loses one power of h from adding $N = O(1/h)$ steps:

$$\tau_{\text{loc}}^{\text{RK4}} = O(h^5), \quad \tau_{\text{glob}}^{\text{RK4}} = O(h^4). \quad (17)$$

4.3 Left Riemann sum

On one sub-interval $[x_i, x_i + h]$, I Taylor-expand f around x_i :

$$\int_{x_i}^{x_i+h} f(x) dx = \int_0^h [f(x_i) + f'(x_i)t + \frac{1}{2}f''(x_i)t^2 + \dots] dt = hf(x_i) + \frac{h^2}{2}f'(x_i) + O(h^3). \quad (18)$$

Left Riemann keeps only $hf(x_i)$, so the local error on one sub-interval is

$$\tau_{\text{loc}}^{\text{Riemann}} = \frac{h^2}{2}f'(x_i) + O(h^3) = O(h^2). \quad (19)$$

Adding up $N = (b - a)/h$ sub-intervals gives a global error of

$$\tau_{\text{glob}}^{\text{Riemann}} = N \cdot O(h^2) = O(h). \quad (20)$$

4.4 Trapezoid

The trapezoid rule on one sub-interval is $(h/2)[f(x_i) + f(x_i + h)]$. I expand $f(x_i + h)$ in Taylor series:

$$f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2}f''(x_i) + \frac{h^3}{6}f'''(x_i) + O(h^4). \quad (21)$$

So the trapezoid estimate becomes

$$\frac{h}{2}[f(x_i) + f(x_i + h)] = hf(x_i) + \frac{h^2}{2}f'(x_i) + \frac{h^3}{4}f''(x_i) + O(h^4). \quad (22)$$

The exact integral I derived above for Riemann already gives me

$$\int_{x_i}^{x_i+h} f(x) dx = hf(x_i) + \frac{h^2}{2}f'(x_i) + \frac{h^3}{6}f''(x_i) + O(h^4). \quad (23)$$

Subtracting, the local error is

$$\tau_{\text{loc}}^{\text{trap}} = \left(\frac{h^3}{4} - \frac{h^3}{6}\right) f''(x_i) + O(h^4) = \frac{h^3}{12}f''(x_i) + O(h^4) = O(h^3). \quad (24)$$

So trapezoid is exact for linear functions (no f' term left, only f''). Adding up $N = (b-a)/h$ sub-intervals gives the global error

$$\tau_{\text{glob}}^{\text{trap}} = N \cdot O(h^3) = O(h^2). \quad (25)$$

4.5 Simpson

Simpson's rule fits a parabola through three consecutive grid points and integrates the parabola exactly. On one pair of sub-intervals $[x_i, x_i + 2h]$ (width $2h$), the rule is

$$\frac{h}{3} [f(x_i) + 4f(x_i + h) + f(x_i + 2h)].$$

I Taylor-expand $f(x_i + h)$ and $f(x_i + 2h)$ around x_i and add them up. After collecting terms, the f, f', f'', f''' contributions all match the exact integral, and the first term that does not cancel involves $f^{(4)}$. Doing the algebra carefully (I did this by hand):

$$\tau_{\text{loc}}^{\text{Simpson}} = -\frac{h^5}{90}f^{(4)}(x_i) + O(h^6) = O(h^5). \quad (26)$$

This is a stronger result than you might expect, because parabolas are only degree 2 but Simpson actually integrates cubics exactly too (the cubic term f''' happens to cancel because of the symmetric 1, 4, 1 weights). Adding up $N/2 = (b-a)/(2h)$ pairs of sub-intervals gives the global error

$$\tau_{\text{glob}}^{\text{Simpson}} = \frac{N}{2} \cdot O(h^5) = O(h^4). \quad (27)$$

4.6 Summary of expected global errors

Method	Local error	Global error
Euler	$O(h^2)$	$O(h)$
RK4	$O(h^5)$	$O(h^4)$
Left Riemann	$O(h^2)$	$O(h)$
Trapezoid	$O(h^3)$	$O(h^2)$
Simpson	$O(h^5)$	$O(h^4)$

I check each of these numerically in Section 6.

5 Code Organization

The code is split into small library files and two main scripts. The two integrator files `ode_integrators.py` and `quad_integrators.py` have one function per method. The two physics files `coupled_oscillator.py` and `drag_stopping.py` contain the right-hand side or integrand, the exact solution, and the physical parameters. The main scripts `main_ode.py` and `main_integral.py` run the experiments and produce all figures in this report. See `README.md` for how to install and run.

6 Validation

I validate my code in three ways: (i) I compare my numerical solutions to the exact solutions and to the scipy versions, (ii) I check two physical properties for each of the two problems, and (iii) I check that both the local and the global numerical errors follow the truncation-error scaling from Section 4.

6.1 ODE: comparison of my methods to the exact solution and scipy

Figure 1 shows $x_1(t)$ over 30 time units with only mass 1 displaced ($x_1(0) = 1$, everything else zero) at step size $h = 0.05$. My RK4 (blue dots) sits on top of the exact curve (black) and the scipy DOP853 reference (green dashed). On this scale you cannot tell them apart. My Euler (red dots) is fine for the first few oscillations but then its amplitude grows a lot, and by $t = 30$ it is about 4 times too big. I explain why this happens in Section 7.

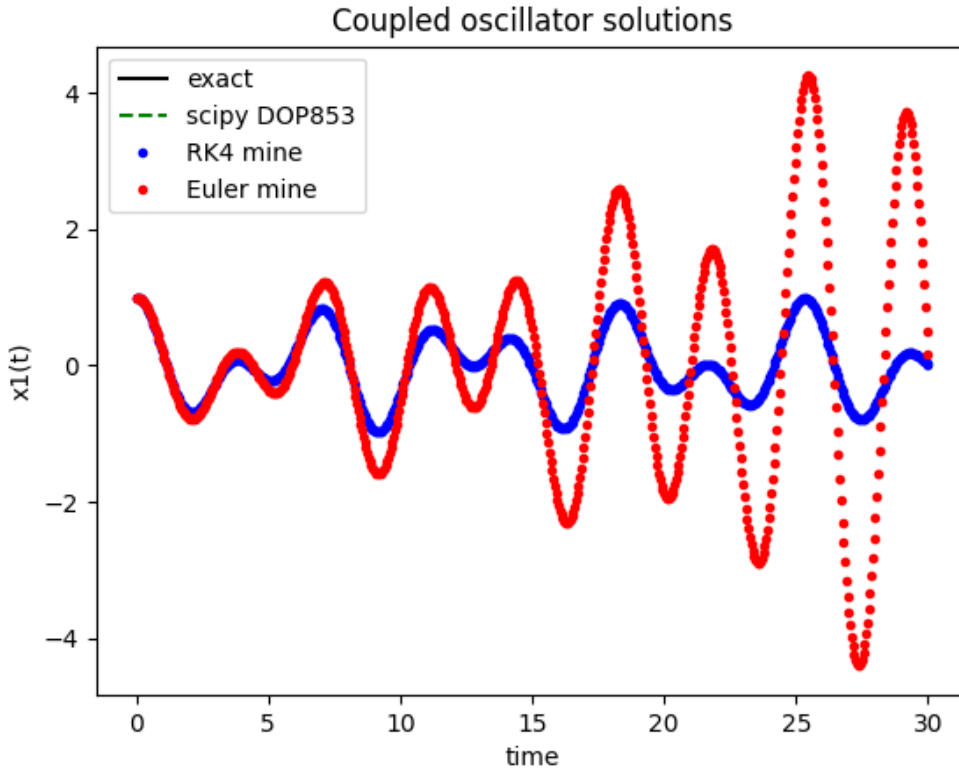


Figure 1: Coupled oscillator solutions with $x_1(0) = 1$, everything else zero, at $h = 0.05$. RK4 (blue) matches the exact curve (black) and scipy DOP853 (green dashed). Euler (red) starts fine but the amplitude grows in time.

6.2 ODE physical property 1: energy conservation

There is no friction, so the total energy from equation (4) should stay constant. Figure 2 plots $E(t)/E(0)$ over 30 time units. Euler grows to about $70E(0)$, which shows the amplitude is growing exponentially in time. RK4 loses only a few parts in 10^6 over the whole run. Neither method

conserves energy exactly, but RK4 is close enough that I need a log plot to see any change.

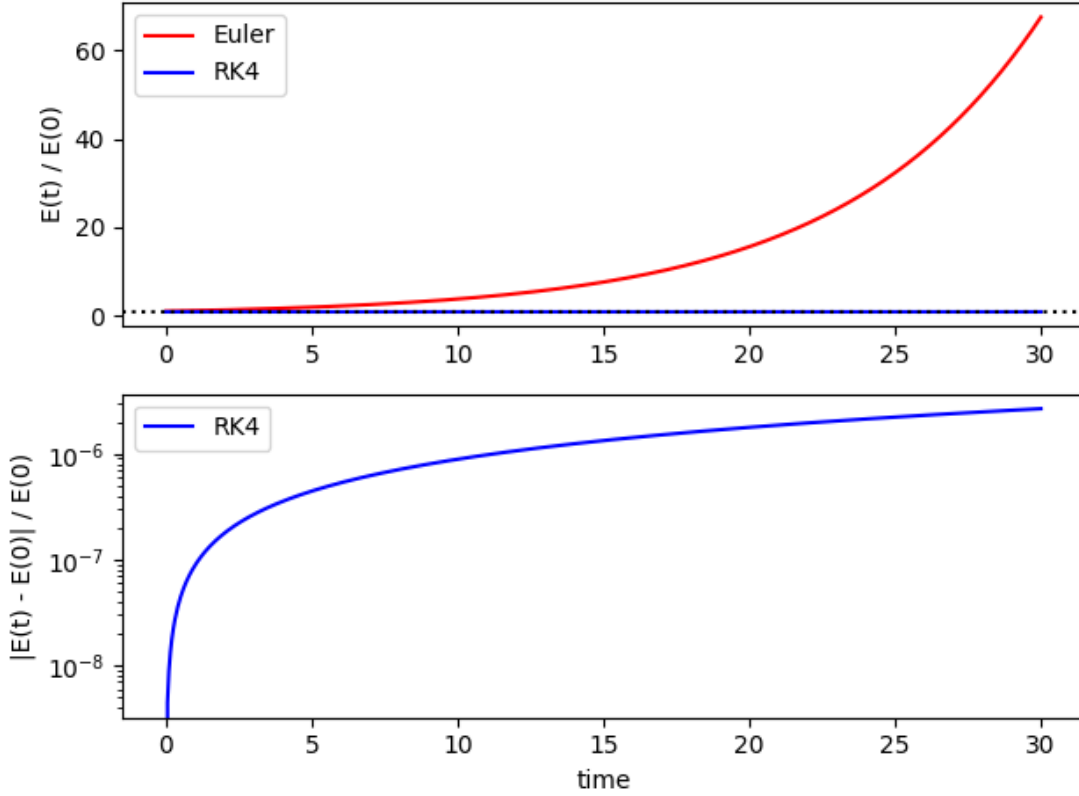


Figure 2: Energy divided by $E(0)$ with $h = 0.05$. Top: linear scale. Euler (red) blows up; RK4 (blue) stays flat. Bottom: log scale of $|E(t) - E(0)|/E(0)$ for RK4 only. Even RK4 has a small drift, but it is around 10^{-6} after 30 time units.

6.3 ODE physical property 2: normal-mode purity and beating

This test has two parts. First, if I start the system in a pure normal mode ($x_1 = x_2$ or $x_1 = -x_2$, both masses at rest), it should stay in that mode forever and oscillate at the single frequency ω_+ or ω_- . When I try this in my code with RK4, this is exactly what happens: the difference $x_1 - x_2$ stays zero if I start with $x_1 = x_2$, and the sum $x_1 + x_2$ stays zero if I start with $x_1 = -x_2$. The single frequency I get by counting oscillations matches equation (3) in both cases.

Second, if I only displace one mass at the beginning, both normal modes are excited with equal amplitude and I get a beating pattern. Using the identity

$$\cos(\omega_+ t) + \cos(\omega_- t) = 2 \cos\left(\frac{\omega_- - \omega_+}{2} t\right) \cos\left(\frac{\omega_- + \omega_+}{2} t\right),$$

the solution is a fast oscillation at the average frequency multiplied by a slow envelope at frequency $(\omega_- - \omega_+)/2$. Figure 3 shows exactly this: the RK4 curve (blue) matches the exact curve (black dashed), and both stay inside the envelope $\pm \cos((\omega_- - \omega_+)t/2)$ (red dotted) for many beat periods.

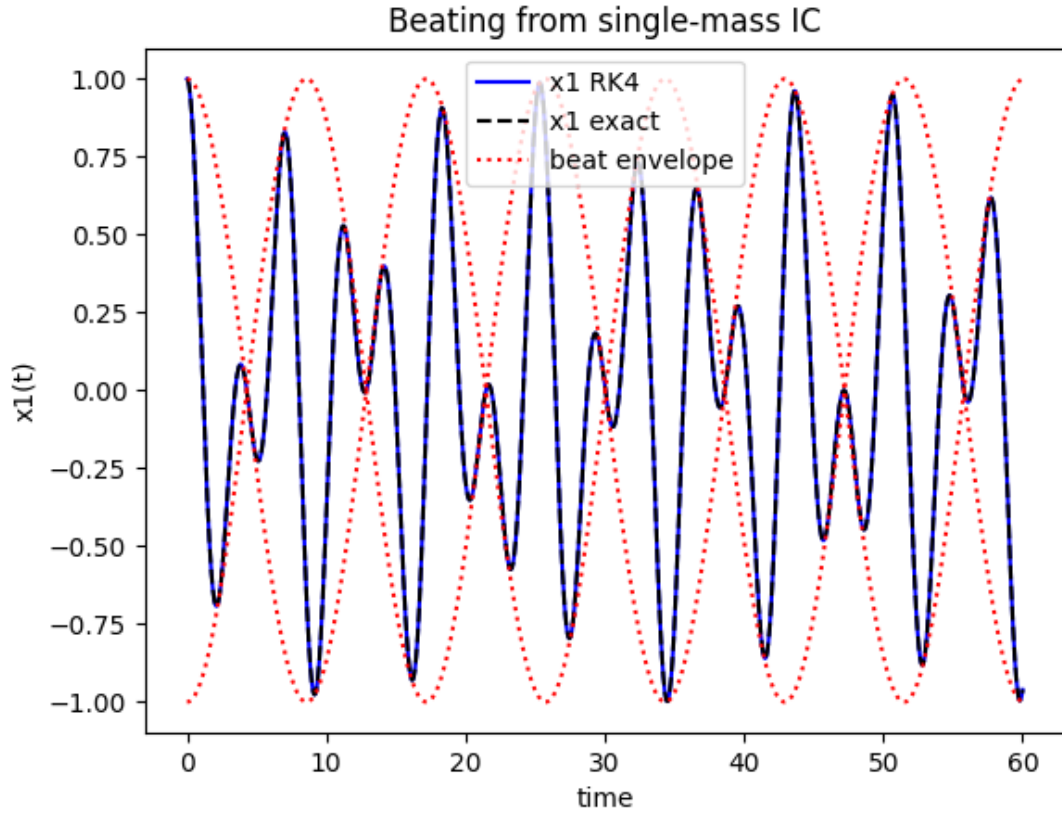


Figure 3: Beating pattern with only mass 1 displaced. The fast oscillation is at the average frequency $(\omega_+ + \omega_-)/2$; the slow envelope (red dotted) is at $(\omega_- - \omega_+)/2$. RK4 (blue) matches the exact solution (black dashed) over 60 time units.

6.4 ODE: global error vs step size

For this test I use $t_{\text{end}} = 5$ (about one third of a beat period). If I use a longer t_{end} , the Euler amplitude drift becomes too big at the coarsest step sizes and I cannot see the clean slope. Figure 4 shows the global error at $t_{\text{end}} = 5$ against step size on log-log axes. From a linear fit, Euler has slope 1.02 and RK4 has slope 3.96, which are very close to the theory values of 1 and 4 from Section 4. The RK4 curve flattens for the smallest h because the error hits the floating-point precision limit of double-precision numbers (around 10^{-14}). I did not include those points in the fit.

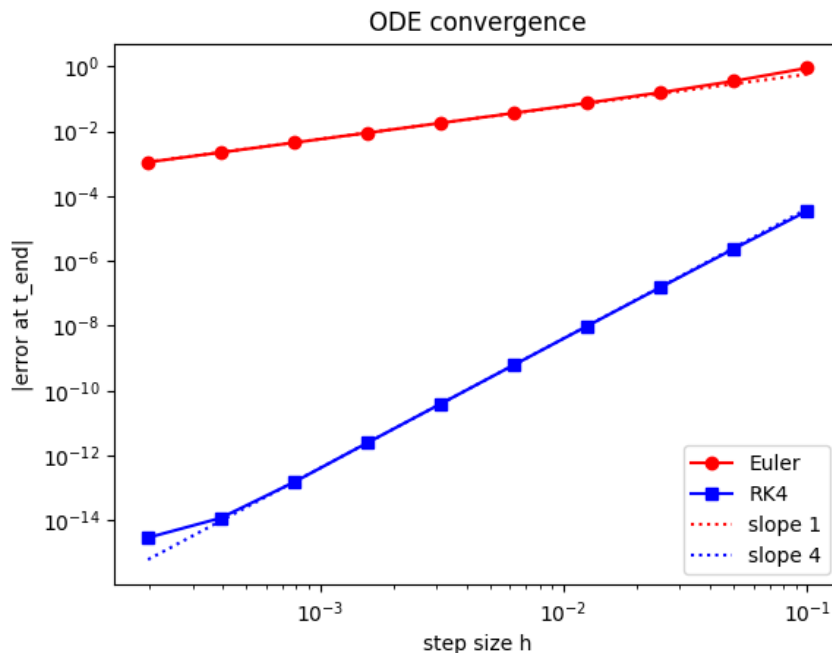


Figure 4: Global error at $t_{\text{end}} = 5$ vs step size h . Fitted slopes are 1.02 for Euler and 3.96 for RK4, matching the theory (1 and 4). RK4 flattens at very small h because of floating-point precision.

6.5 ODE: local (one-step) error

The convergence test above shows the global error slope. The assignment also asks me to check the local (one-step) error scaling. From Section 4, theory says Euler has local error $O(h^2)$ and RK4 has $O(h^5)$. I take one step of each method from $t = 0$ with different h , compare to the exact solution at that same h , and plot the error on log-log axes. Figure 5 shows the result. The fitted slopes are 1.99 for Euler and 4.99 for RK4, which match the theory values of 2 and 5.

6.6 Integral: comparison of my rules to the exact answer and scipy

For a first test of the quadrature rules I integrate equation (6) with $v_0 = \tau = 1$ and $T = 3$ using $n = 32$ sub-intervals. The exact answer from equation (7) is $x_{\text{exact}} = 1 - e^{-3} \approx 0.9502$.



Figure 5: Local (one-step) error of Euler and RK4 vs step size h . Dotted lines show slopes 2 and 5 for comparison. Fitted slopes: Euler 1.99, RK4 4.99, matching the local truncation error from Section 4.

Rule	Estimate	Error
Left Riemann (mine)	0.9954500176	$+4.52 \times 10^{-2}$
Trapezoid (mine)	0.9509087864	$+6.96 \times 10^{-4}$
Trapezoid (scipy)	0.9509087864	$+6.96 \times 10^{-4}$
Simpson (mine)	0.9502133390	$+4.07 \times 10^{-7}$
Simpson (scipy)	0.9502133390	$+4.07 \times 10^{-7}$

My trapezoid and Simpson give exactly the same numbers as the scipy versions, which makes sense because they use the same formula on the same grid. Figure 6 shows the integrand with the grid points I used.

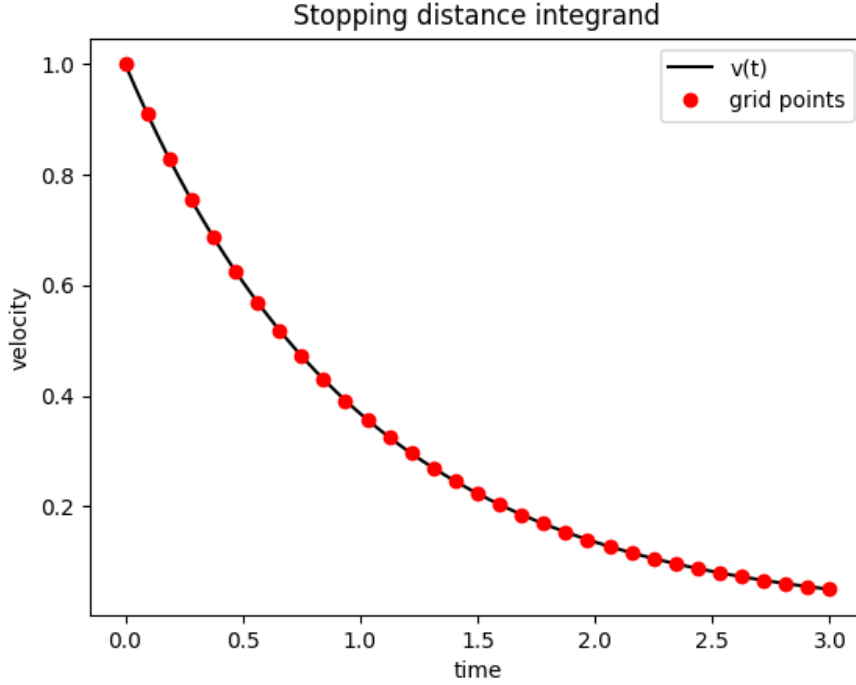


Figure 6: The integrand $v(t) = v_0 e^{-t/\tau}$ with the $n = 32$ grid points (red) I used for the comparison.

6.7 Integral physical properties: short-time and long-time limits

1. **Short-time limit** ($T \ll \tau$). If I expand $e^{-T/\tau} \approx 1 - T/\tau + \frac{1}{2}(T/\tau)^2$, I get $x(T) \approx v_0 T - v_0 T^2/(2\tau)$. So the leading term is $v_0 T$ (like a free object without drag), and the correction is quadratic in T . The right panel of Figure 7 shows the fractional deviation from $v_0 T$. It starts at zero and grows almost linearly with T , exactly like the correction predicts.
2. **Long-time limit** ($T \rightarrow \infty$). The object never fully stops, but the total distance it travels is finite. From equation (7), as $T \rightarrow \infty$ the exponential goes to zero and $x_\infty = v_0 \tau$. The left panel of Figure 7 shows $x(T)$ approaches $v_0 \tau$ for $T \gtrsim 5\tau$.

6.8 Integral: global error vs step size

Figure 8 shows the absolute error of each rule against step size on log-log axes. The fitted slopes are 1.02 for Riemann, 2.00 for trapezoid, and 3.99 for Simpson (I skipped the last few Simpson points

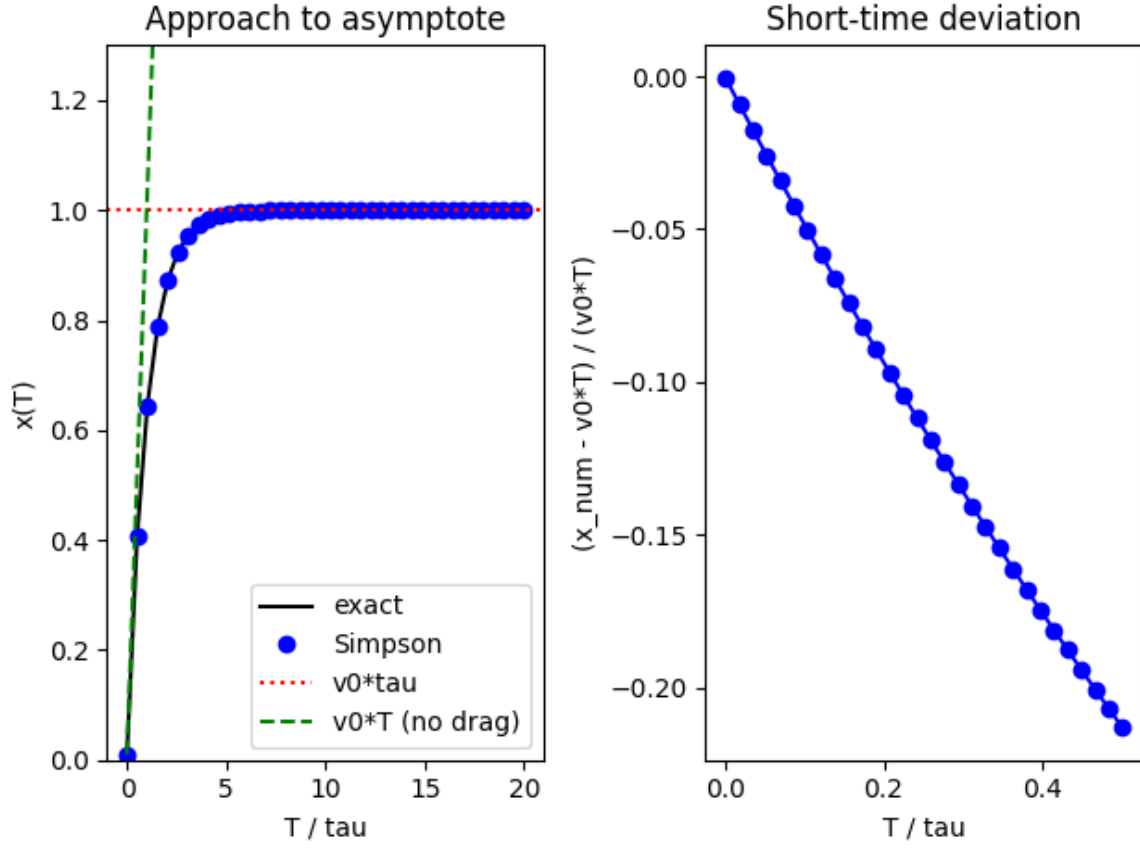


Figure 7: Physical limits of the stopping distance. Left: $x(T)$ vs T/τ with the two limits overlaid. Simpson (blue dots) matches the exact curve (black). Right: fractional deviation from $v_0 T$ at short times, growing linearly like the expansion predicts.

where the error hits the floating-point limit). All three slopes match the theory from Section 4. My trapezoid and Simpson points are on top of the scipy ones, which is expected because they use the same formula.

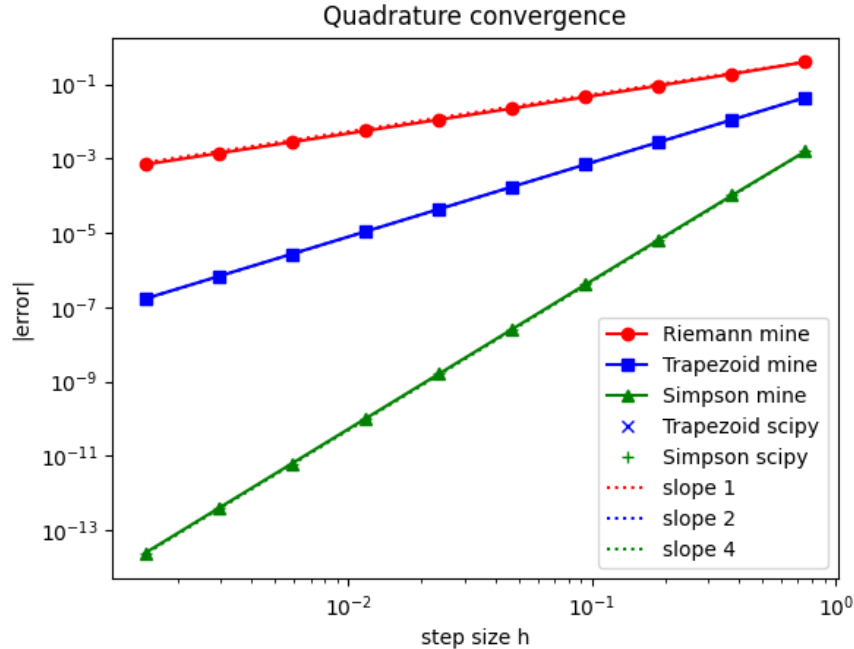


Figure 8: Absolute error of each rule vs step size h , for the stopping-distance integrand with $v_0 = \tau = 1$ and $T = 3$. Dotted lines show slopes 1, 2, 4 for comparison. Fitted slopes: Riemann 1.02, trapezoid 2.00, Simpson 3.99. Simpson flattens at small h due to floating-point limits.

6.9 Integral: local (one sub-interval) error

Same idea as Section 6.5 but for the quadrature rules. From Section 4, theory says local error goes like h^2 for Riemann, h^3 for trapezoid, and h^5 for Simpson. I apply each rule to one sub-interval of width h (or one pair $2h$ for Simpson since it needs an even number of sub-intervals) and compare to the exact integral over that small interval. Figure 9 shows the result. The fitted slopes are 1.97 for Riemann, 2.95 for trapezoid, and 4.90 for Simpson, which match the theory values of 2, 3, 5.

7 Where My Code Is Worst

The worst place my code performs is **Euler on the coupled oscillator**. As shown in Figure 1, at $h = 0.05$ over 30 time units, the Euler amplitude grows to about 4 times the true value. Figure 2 shows the energy grows to about 70 times its starting value. Meanwhile RK4 at the same step size stays within 10^{-6} of the exact answer.

The reason is that Euler always adds a small extra amount to the amplitude at every step when the system is oscillating. This is a known issue: Euler is not stable for undamped oscillators. Each step, the amplitude gets multiplied by a factor slightly bigger than 1, so after many steps the amplitude grows exponentially in time. That is why we see the energy grow to $70E(0)$ instead of staying at $E(0)$.

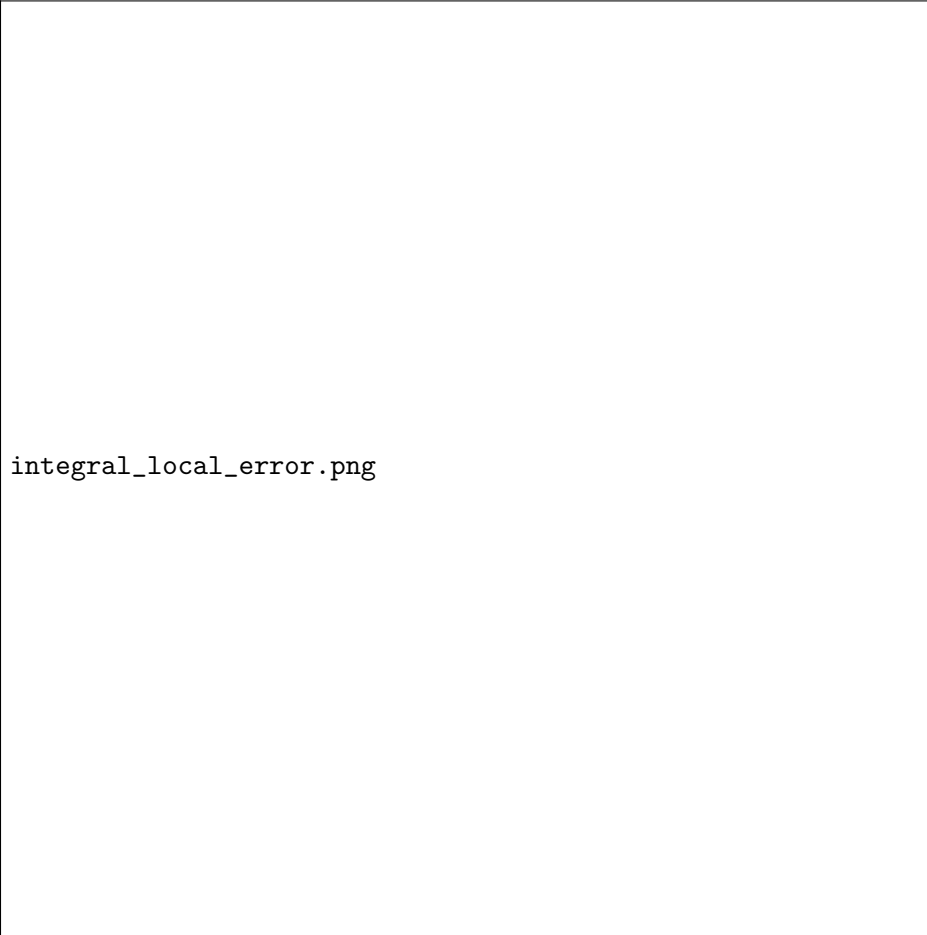


Figure 9: Local (one sub-interval) error of each quadrature rule vs step size h . Dotted lines show slopes 2, 3, 5 for comparison. Fitted slopes: Riemann 1.97, trapezoid 2.95, Simpson 4.90, matching the local truncation error from Section 4.

This is a problem with Euler, not with my code. For problems like this, better methods should be used instead (RK4 works fine). This is also why in Section 6.4 I used a shorter $t_{\text{end}} = 5$ for the convergence test, because for short times Euler is not too unstable and I can still see the correct slope-1 scaling.

A smaller problem shows up in the quadrature test (Figure 8): for very small h , Simpson's error stops decreasing and flattens at around 10^{-14} . When h gets very small, the round-off error from the computer arithmetic becomes as big as the truncation error, so making h smaller does not help anymore. It is not a bug either, it is a limit of the arithmetic.

8 Conclusion

I wrote five integrators by hand: Euler and RK4 for the ODE, and Riemann, trapezoid, and Simpson for the definite integral. I tested them on two mechanics problems that have exact solutions. All five observed global error slopes and all five local error slopes match the theory values from Section 4 very closely (within 0.1). I also checked two physical properties for each problem: energy conservation and normal-mode beating for the oscillator, and the short-time and long-time limits for the stopping distance. The worst behavior is Euler on the oscillator, where the amplitude and energy grow in time. This is a known issue with Euler on oscillators and not a bug in my code.